



Gestión de Datos y Estructuras Estáticas

Objetivo: Aplicar los conceptos de objetos de datos, tipos de datos elementales y estructurados (arreglos, matrices, registros y cadenas) bajo el estándar de Lenguaje C, resolviendo desafíos de entornos de producción real mediante pensamiento lógico y auditoría de código.

Ejercicio 1: Telemetría de Producción y Auditoría de Peso de Piezas

Consigna: Has sido contratado por una planta automatizada de autopartes. El sistema de telemetría controla el peso (en gramos) de las piezas que salen de la línea de producción. Para auditar un lote, debes capturar un arreglo con el peso de 20 piezas consecutivas. Tu misión es desarrollar el módulo de auditoría de calidad.

Requerimientos: Codificar un programa estructurado en funciones que permita:

Detección de Falla Crítica: Implementar una función que reciba el arreglo y determine si existe al menos una pieza con peso nulo (0), lo que indicará una falla inmediata del sensor de pesaje.

Auditoría de la Estación A: Realizar un recorrido secuencial utilizando exclusivamente aritmética de índices para mostrar únicamente los pesos almacenados en posiciones (índices) pares, correspondientes a las piezas inspeccionadas por la estación de control A.

Análisis del Flujo de Datos: Calcular e informar la cantidad de pesos pares contenidos, imprimiendo en pantalla una explicación en formato de comentario sobre cómo el compilador diferencia el valor del dato (Valor-R) de su ubicación física en la memoria (Valor-L).

Picos de Producción: Determinar el peso máximo y el peso mínimo registrados en el lote, informando en qué índice exacto del arreglo se encuentra cada uno.

Rendimiento Promedio: Calcular e informar el peso promedio de las 20 piezas del lote analizado.

Misión Optimización del Stack: Demostrar en el informe de consola si el identificador del arreglo se comporta como un puntero constante a su primer elemento (*Array Decay*) y calcular el tamaño total en bytes del arreglo utilizando `sizeof`.

Ejercicio 2: Procesamiento Meteorológico de Temperaturas

Consigna: Estás a cargo del software de procesamiento de datos de una estación meteorológica provincial. El sistema registra las temperaturas máximas diarias (°C) durante 15 días continuos. Tu objetivo es procesar las tendencias y activar las alarmas climáticas.

Requerimientos: Codificar un programa que permita:

a) Módulo de Captura: Implementar una función parametrizada que permita la carga segura de dicha información en un arreglo unidimensional.

b) Rastreo Climático: Implementar una función de búsqueda secuencial que, dada una temperatura ingresada por el



usuario, informe si existió y en qué día específico (índice) se registró.

c) Reordenamiento de Datos (Fase Alfa): Generar un arreglo auxiliar en memoria, clonando los datos del primero, e implementar el algoritmo de ordenamiento burbuja mejorado para ordenar las temperaturas de forma ascendente, mostrando el resultado final.

d) Reordenamiento Dinámico (Fase Beta): Implementar una función modular avanzada que realice el ordenamiento de forma descendente, reutilizando la lógica anterior mediante la parametrización del criterio de orden.

e) Línea de Base: Calcular e informar la temperatura promedio registrada durante el período de 15 días.

f) Alerta de Emergencia: Informar la cantidad de días en que la temperatura máxima superó el umbral crítico de los 30 °C, emitiendo el mensaje destacado "ALERTA DE OLA DE CALOR" si la cantidad es mayor a 3.

g) Misión Eficiencia: Implementar una función de Búsqueda Binaria sobre el arreglo ordenado de forma ascendente, imprimiendo cuántas comparaciones le tomó al procesador encontrar el dato o determinar su ausencia.

Ejercicio 3: Auditoría Financiera de Librerías

Consigna: Eres el auditor financiero de una cadena de librerías. La empresa almacena en una matriz de 4 x 4 las unidades vendidas por 4 sucursales distintas durante los 4 trimestres del año. Tu supervisor te pide analizar el rendimiento aplicando las reglas de almacenamiento de C.

Atendiendo al esquema de almacenamiento **Row-Major Order** (ordenación por filas por defecto en C), diseña un programa con funciones que permita:

Requerimientos: Atendiendo al esquema de almacenamiento *Row-Major Order* (ordenación por filas por defecto en C), diseñar un programa con funciones que permita:

a) Inspección Trimestral: Diseñar una función que, solicitando un trimestre específico (fila) por teclado, muestre en una línea limpia las ventas de las 4 sucursales en ese período.

b) Rendimiento Estacional: Diseñar una función que reciba la matriz y devuelva un arreglo con la sumatoria de cada una de las filas (total vendido por trimestre).

c) Rendimiento Corporativo: Diseñar una función que calcule y muestre la sumatoria de cada una de las columnas (total acumulado anual por sucursal).

d) Eje de Control: Calcular el promedio ponderado de los valores almacenados exclusivamente en la diagonal principal de la matriz.

e) Auditoría de Metas: Indicar cuántas celdas de la matriz superan un valor umbral ingresado por teclado por el gerente general.

Ejercicio 4: Control de Calificaciones Académicas

Consigna: Automatización del acta de calificaciones de una comisión de 8 estudiantes, donde cada uno rinde 4 evaluaciones parciales obligatorias durante el cuatrimestre (notas enteras de 0 a 10) representadas en una matriz de 8x4.

Requerimientos: Codificar un programa que use funciones para:

a) Carga del Acta: Cargar la matriz con las notas de los parciales para cada estudiante (por teclado o generación aleatoria controlada de 0 a 10).

b) Diagnóstico de Condición Académica: Calcular e informar el promedio individual de cada estudiante y determinar su estado: Promocionado (promedio > 7), Regular (promedio entre 4 y 6.99) o Libre (promedio < 4).

- c) Auditoría del Examen:** Calcular e informar el promedio general de la comisión en cada uno de los 4 parciales de forma independiente, diagnosticando en qué examen hubo mejor y peor rendimiento.
- d) Récorde de la Comisión:** Determinar la nota más alta y más baja de toda la matriz, indicando a qué estudiante y parcial corresponde cada una.
- e) Informe Estadístico:** Informar la cantidad total de estudiantes en condición de "Libre".
- f) Misión Integridad de Datos:** Implementar una función de validación que exija el reingreso inmediato si se ingresa una nota menor a 0 o mayor a 10, registrando la cantidad de errores cometidos.

Ejercicio 5: Backend de Control de Matrículas Universitarias

Consigna: El departamento de Informática de una universidad te solicita diseñar el backend para el control de matrículas. Se gestiona el dictado de 4 cursos de vanguardia: “1 – IA Generativa”, “2 – Administración de Base de Datos”, “3 – Cableado de Datos” y “4 – Gestión Inteligente de APs”. Los cursos se dictan en 3 turnos: “1 – Mañana”, “2 – Tarde” y “3 – Vespertino”.

Diseña una solución donde el ingreso de transacciones sea dinámico (Curso, Turno, Cantidad de inscriptos) y concluya cuando se ingrese un código de curso igual a 0.

Requerimientos: Procesar la matriz resultante para:

- a) Censo Estudiantil:** Informar la cantidad total absoluta de alumnos inscriptos en toda la oferta académica.
- b) Foco de Curso:** Dado un número de curso por teclado, indicar qué turno tiene la mayor densidad de inscriptos y detallar el total de asistentes a ese curso.
- c) Foco de Turno:** Dado un número de turno por teclado, indicar qué curso es el más demandado y la cantidad total de alumnos en ese horario.
- d) El Líder de la Oferta:** Informar el nombre del curso con mayor cantidad total de inscriptos, consolidando sus tres turnos.
- e) Franja Horaria Crítica:** Informar el turno con mayor cantidad total de inscriptos, consolidando los cuatro cursos.

Ejercicio 6: Gestión de Padrón Estudiantil

Consigna: La Secretaría Académica requiere migrar sus fichas de papel a estructuras de datos en C. Utilizando la instrucción typedef para crear un alias de estructura limpia, define un tipo de dato t_alumno que encapsule: Nombre (cadena de 30), Apellido (cadena de 30) y DNI (entero largo).

Requerimientos: Codificar un programa que use funciones para:

- a) Captura del Padrón:** Declarar un arreglo estático de registros para 10 alumnos y realizar la carga controlada.
- b) Interfaz Tabular:** Implementar un subprograma de listado que proyecte los datos formateados simulando una hoja de cálculo con columnas alineadas.
- c) Filtro Generacional:** Informar la cantidad de alumnos cuyo DNI sea superior a 45.000.000.
- d) Indexación Alfabética:** Ordenar el arreglo completo alfabéticamente por el campo Apellido intercambiando las estructuras en memoria.
- e) Motor de Búsqueda:** Recuperar e informar Nombre y Apellido dado un DNI ingresado por teclado (o mostrar error si no existe).
- f) Misión Auditoría de Tipado:** Diseñar una estructura gemela t_aspirante, intentar la asignación directa alumno1 = aspirante1; y explicar el concepto de Equivalencia Nominal de Tipos en C.

Ejercicio 7: Motor de Filtrado y Segmentación Demográfica

Consigna: Una agencia de empleo internacional te encomienda el desarrollo de su motor de filtrado de perfiles para una convocatoria masiva. El sistema procesa los datos de 50 postulantes. Para cada uno se debe registrar en una estructura heterogénea: Nombre, Sexo ('M'/'F'), DNI, Edad y Código de Localidad (valores enteros acotados del 1 al 19).

Requerimientos: Desarrollar una solución basada en subprogramas para:

- a) **Segmentación de Talento:** Indicar la cantidad exacta de postulantes del sexo femenino menores de 30 años.
- b) **Métrica Demográfica:** Calcular e informar la edad promedio de los postulantes de sexo masculino.
- c) **Filtro Geográfico:** Listar los nombres y DNI de los postulantes que residan en la localidad 14.
- d) **El Veterano del Grupo:** Informar el nombre completo y DNI del postulante de mayor edad.
- e) **Escalabilidad por Edad:** Generar un listado completo de postulantes ordenados por edad de forma ascendente.
- f) **Misión Validación de Rango:** Validar que el Código de Localidad pertenezca estrictamente al rango cerrado [1, 19] mediante un bucle de control.

Ejercicio 8: Control de Inventario Mayorista

Consigna: El depósito central de un distribuidor mayorista necesita optimizar la velocidad de su inventario. Administran una estructura estática de 50 productos, donde cada ítem está definido por: Código (entero), Precio de Costo (float) y Stock (entero).

Requerimientos: Escribir un programa que usando funciones implemente:

- a) **Ordenamiento por Selección:** Ordenar el arreglo completo por Código en orden ascendente usando el método de Selección.
- b) **Búsqueda Binaria Corregida:** Recuperar el stock actual de un producto mediante Búsqueda Binaria según su código.
- c) **Valorización Activa:** Calcular e informar la inversión retenida (Precio de costo x Stock) por cada producto de forma individual.
- d) **Producto Premium:** Localizar e informar el producto con el mayor precio de costo.
- e) **Auditoría Contable:** Calcular e informar el valor financiero total invertido en la totalidad del inventario.
- f) **Misión Control de Quiebre:** Generar un reporte de "Alerta de Desabastecimiento" listando los códigos de productos cuyo stock sea exactamente 0.

Ejercicio 9: Procesamiento de Cadenas de Caracteres

Consigna: Trabajas en el equipo de ciberseguridad y procesamiento de textos de una suite ofimática. Debes validar y transformar cadenas de caracteres bajo el estricto estándar de C, donde las strings son arreglos que finalizan con el delimitador o centinela nulo ('\0').

Lee desde el teclado la frase inicial fija: "Programación Procedural 2025".

Requerimientos: Elaborar un programa que usando funciones permita:

- a) **Inyección en Memoria:** Reemplazar el carácter '5' por '6' mediante aritmética de índices en el arreglo.
- b) **Extracción Extrema:** Copiar la palabra "Procedural" en una nueva variable de cadena independiente usando strncpy de <string.h>.
- c) **Auditoría de Vocales:** Contar de forma manual las vocales inspeccionando directamente el centinela '\0' (sin funciones

de asistencia).

d) Normalización: Convertir la frase completa a mayúsculas iterando con `toupper()` de `<ctype.h>`.

e) Criptografía Espejo: Generar y mostrar una nueva cadena con la frase totalmente invertida.

f) Misión Desbordamiento Protegido: Reemplazar `gets()` por `fgets()` y explicar en un comentario cómo esta función previene un ataque de *Buffer Overflow*.

Ejercicio 10: Validador de Seguridad

Consigna: Trabajas en el equipo de ciberseguridad y procesamiento de textos de una suite ofimática. Debes validar y transformar cadenas de caracteres bajo el estricto estándar de C, donde las strings son arreglos que finalizan con el delimitador o centinela nulo (`'\0'`).

Lee desde el teclado la frase inicial fija: "Programación Procedural 2025".

Requerimientos: Elaborar un programa modular que use funciones para:

a) Conteo Nativo (Anti-Librería): Calcular la longitud de cualquier cadena recorriéndola manualmente celda por celda hasta `'\0'` (prohibido usar `strlen`).

b) Validador Policial de Passwords: Validar con `<ctype.h>` si la contraseña contiene al menos una letra mayúscula y al menos un dígito numérico.

c) Tokenizador de Frases: Fragmentar la frase de recuperación en palabras individuales con `strtok()` de `<string.h>` y almacenarlas en una matriz de caracteres.

d) Analizador Capicúa: Determinar iterativamente si una palabra extraída es palíndromo comparando desde los extremos hacia el centro.

e) Módulo de Enmascaramiento: Localizar subcadenas con `strstr()` de `<string.h>` y reemplazarlas por otra cadena del mismo tamaño para ofuscar datos sensibles (ej. reemplazar "clave" por "*****").

f) Misión Limpieza de Buffer: Purgar el salto de línea residual `'\n'` atrapado en `stdin` al combinar lecturas numéricas con `fgets()` utilizando `while(getchar() != '\n');`.

Ejercicio 11: Sistema de Punto de Venta

Consigna: Estás diseñando el software para el punto de venta (POS) de un comercio mayorista. El almacén administra una base de datos estática de 250 artículos definidos estructuralmente por: Código, Nombre (cadena), Precio Unitario (float) y Stock (entero).

Desarrolla el flujo del negocio dividiendo el programa en dos fases obligatorias:

- **Fase Inicial (Carga):** Se debe poblar el catálogo completo de los 250 artículos con datos válidos.
- **Fase Operativa (Procesamiento de Ventas):** Implemente la carga continua de transacciones de venta (ingresando el Nombre del artículo a vender y la Cantidad solicitada) de forma interactiva, hasta que el operador escriba la palabra clave "FIN" en el nombre.

Requerimientos: Elaborar un programa que usando funciones responda a lo siguiente:

a) Despacho e Integridad: Buscar por nombre, descontar stock si hay disponibilidad e imprimir "DESABASTECIMIENTO" si el stock resultante queda en 0.

b) Caja Registradora: Calcular e informar el monto total monetario recaudado durante la jornada.

c) El Producto Estrella: Identificar el artículo con la mayor cantidad de unidades vendidas.

d) Alerta de Reposición Crítica: Listar los artículos cuyo stock residual sea menor a 5 unidades para emisión de orden de compra.



f) Misión Búsqueda Optimizada: Implementar la comparación eficiente de cadenas mediante `strcmp()` de `<string.h>` e informar un error inmediato si el producto solicitado no existe.

Ejercicio 12: Arquitectura Modular de Proveedores Logísticos

Consigna: Has sido ascendido a Arquitecto de Software de la distribuidora. Tu misión es conectar el módulo de inventario de productos con la base de datos de los proveedores logísticos para evaluar el desempeño de la cadena de suministro. El sistema gestiona una nómina fija de 22 proveedores especializados.

Requerimientos: Elaborar un programa que usando funciones permita resolver los siguientes ítems:

- a) Registro Logístico:** Almacenar la información de proveedores validando el rango entero correlativo de 1000 a 1022.
- b) Vinculación Transaccional:** Conectar con el inventario del Ejercicio 8 para actualizar el stock e incrementar contadores de transacciones en tiempo real.
- c) Filtro de Alta Performance:** Generar en tiempo de ejecución una nueva estructura (arreglo de registros dinámico/auxiliar) con proveedores que hayan superado las 10 compras exitosas y mostrarla ordenada alfabéticamente.
- d) Proveedor del Año:** Informar con estadísticas completas al proveedor con mayor actividad de suministro.
- e) Ratio de Inactividad:** Calcular el porcentaje de proveedores ineficientes (aquellos que no alcanzaron las 10 compras).

Ejercicio 13: Arreglos Multidimensionales (Estructura 3D / Cubo)

Consigna: Una empresa de logística gestiona un depósito pequeño representado mediante un módulo tridimensional (cubo) compuesto por **2 niveles/caras**, donde cada nivel consta de una matriz de **3 filas y 3 columnas** de compartimentos de almacenamiento ($2 \times 3 \times 3$).

En cada compartimento se almacena la cantidad de paquetes depositados (números enteros positivos).

Desarrollar un programa en **Lenguaje C estándar** que, mediante el uso de constantes globales (`#define CARAS 2`, `#define FILAS 3`, `#define COLUMNAS 3`) y funciones óptimas, realice las siguientes actividades:

Requerimientos: Codificar un programa que use funciones para:

Carga de Datos: Codificar una función para realizar la carga completa de las cantidades de paquetes en el arreglo tridimensional.

Recorrido por Cara (Nivel Específico): Ingresar desde el programa principal (main) un número de cara/nivel (0 o 1). Desarrollar una función que reciba dicho número y muestre en formato matricial (3×3) únicamente la cara seleccionada, calculando e informando el total de paquetes presentes en esa cara.

Recorrido de la Estructura Completa: Desarrollar una función que recorra la totalidad del cubo ($2 \times 3 \times 3$) y devuelva al main la cantidad total acumulada de paquetes en todo el depósito, junto con la cantidad total de compartimentos vacíos (con valor 0).

Hacer seguimiento utilizando el siguiente lote de prueba:

- **Dimensiones:** CARAS = 2, FILAS = 3, COLUMNAS = 3

Cara 0:

10	0	5
0	20	15
8	12	0

Cara 1:

5	15	0
30	0	10
0	25	5